NATIONAL ECONOMICS UNIVERSITY

FACULTY OF MATHEMATICAL ECONOMICS

Final Project:

Database Management System

RESTAURANT MANAGEMENT SYSTEM

Instructor:
Dr. Vu Duc Minh

Group Members:

1. Vu Dieu Linh - 11245898
2. Nguyen Khanh Ly - 11245902
3. Nguyen Thu Minh - 11245908
4. Cai Bao Ngan - 11245913
5. Le Ngoc Linh Nhi - 11245922

Hanoi, 2026

Abstract

This project presents the design, implementation, and deployment of a comprehensive Restaurant Management System utilizing a relational database backend (MySQL hosted on Aiven Cloud) and an interactive web front-end application built with Python and Streamlit. The system optimizes core operational restaurant features, including table reservations, point-of-sale data management, real-time analytics, and role-based staff authentication. By migrating from traditional file systems to a robust database structure, this project ensures data integrity, security, and scalability for real-world restaurant operations.

Contents

Chapter 1

Introduction

1.1 Project Objective

The primary objective of this project is to design, implement, and deploy a comprehensive digital Restaurant Management System. This system aims to digitize and streamline traditional manual restaurant operations, providing a centralized platform for managing customer relationships, table reservations, food menus, and billing processes. By integrating a robust Relational Database Management System (RDBMS) with an interactive Graphical User Interface (GUI), the project seeks to enhance service efficiency, ensure data integrity, and provide actionable analytics for restaurant administrators to make data-driven decisions.

1.2 System Analysis and Requirements

To fulfill the operational needs of a modern restaurant, the system is logically divided into four core functional modules, each addressing specific business requirements:

1.2.1 Customer Management

The system must maintain a comprehensive database of customer profiles, capturing essential details such as names, phone numbers, emails, and physical addresses. Furthermore, it must incorporate a loyalty program that automatically tracks reward points and assigns membership tiers (e.g., Standard, Gold, Platinum) based on customer interactions. Staff should be able to instantly search for existing customers via phone numbers or register new walk-in guests seamlessly.

1.2.2 Table & Reservation Management

Effective spatial management is crucial for restaurant operations. The system must monitor real-time table statuses (Available, Reserved, Occupied) along with their seating capacities. For reservations, the application needs to validate table availability, ensure the guest count does not exceed table capacity, and link the booking to specific customer profiles. Automated database triggers are required to instantly update a table's status to 'Reserved' upon successful booking.

1.2.3 Menu Management

The application must organize food and beverage offerings into logical categories (e.g., Main Course, Dessert, Drink). For each menu item, the system needs to track its price, cost price, and real-time availability (in-stock or out-of-stock). Security protocols must ensure that only authorized personnel with 'Admin' roles can add new dishes or modify pricing and availability statuses.

1.2.4 Billing and Reporting

The system must facilitate a smooth checkout process capable of handling multiple order types (Dine-in, Takeaway, Delivery). It needs to dynamically calculate line subtotals and grand totals for multiple dishes per invoice. Additionally, the system requires a reporting module restricted to administrators, generating real-time insights such as daily revenue trends, total customer visits, and identifying top-selling dishes using optimized database Views.

1.3 Technology Stack

To achieve high performance, scalability, and security, the project is developed using a modern technology stack that strictly adheres to the course requirements:

1.3.1 Database Management System: MySQL (Aiven Cloud)

MySQL is utilized as the core database engine to ensure ACID (Atomicity, Consistency, Isolation, Durability) compliance and handle complex relational queries. Instead of a local deployment, the database is hosted entirely on **Aiven Cloud**. This cloud-based approach allows 24/7 remote accessibility, enabling multiple staff members to interact with the database concurrently from any location without IP restrictions.

1.3.2 Programming Language & Framework: Python & Streamlit

The application's backend logic is written in **Python**, utilizing the `mysql-connector-python` library to establish secure connections with the cloud database. To satisfy the requirement for an interactive interface, **Streamlit**—an open-source Python framework—is deployed to construct a responsive, web-based GUI. This completely eliminates the need for archaic console-based interactions. Furthermore, built-in Python libraries such as `hashlib` are implemented for SHA-256 password encryption, and `pandas` is used to manipulate and display database records dynamically as dataframes.

Chapter 2

Database Design and Implementation

2.1 Business Rules

Before constructing the Entity-Relationship Diagram, specific business rules were established based on the operational requirements of the restaurant. These rules define the entities, their constraints, and how they interact with one another:

- **Employees & Operations:** The restaurant employs multiple staff members with specific roles (admin, cashier, waiter). An employee can process multiple invoices, but each invoice is processed by exactly one employee.
- **Customer Management:** A customer can make multiple reservations and have multiple invoices over time. Customers accumulate reward points based on their transactions, which determine their membership tier (Standard, Gold, Platinum).
- **Menu & Categories:** The menu is divided into several categories. Each category contains multiple menu items (dishes or beverages), but a specific menu item belongs to only one category.
- **Reservations:** A single booking (reservation) is made by one customer. To accommodate large groups, one reservation can include multiple tables, and a table can be booked for multiple reservations at different times.
- **Billing & Orders:** An invoice records a specific transaction (Dine-in, Takeaway, or Delivery). It contains multiple menu items, and each menu item can appear on multiple invoices.

2.2 Entity-Relationship (ER) Diagram

Based on the defined business rules, an Entity-Relationship Diagram (ERD) was designed to visually represent the logical structure of the database. The ERD illustrates the primary entities, their attributes, and the cardinalities (1:M and M:N relationships) between them.

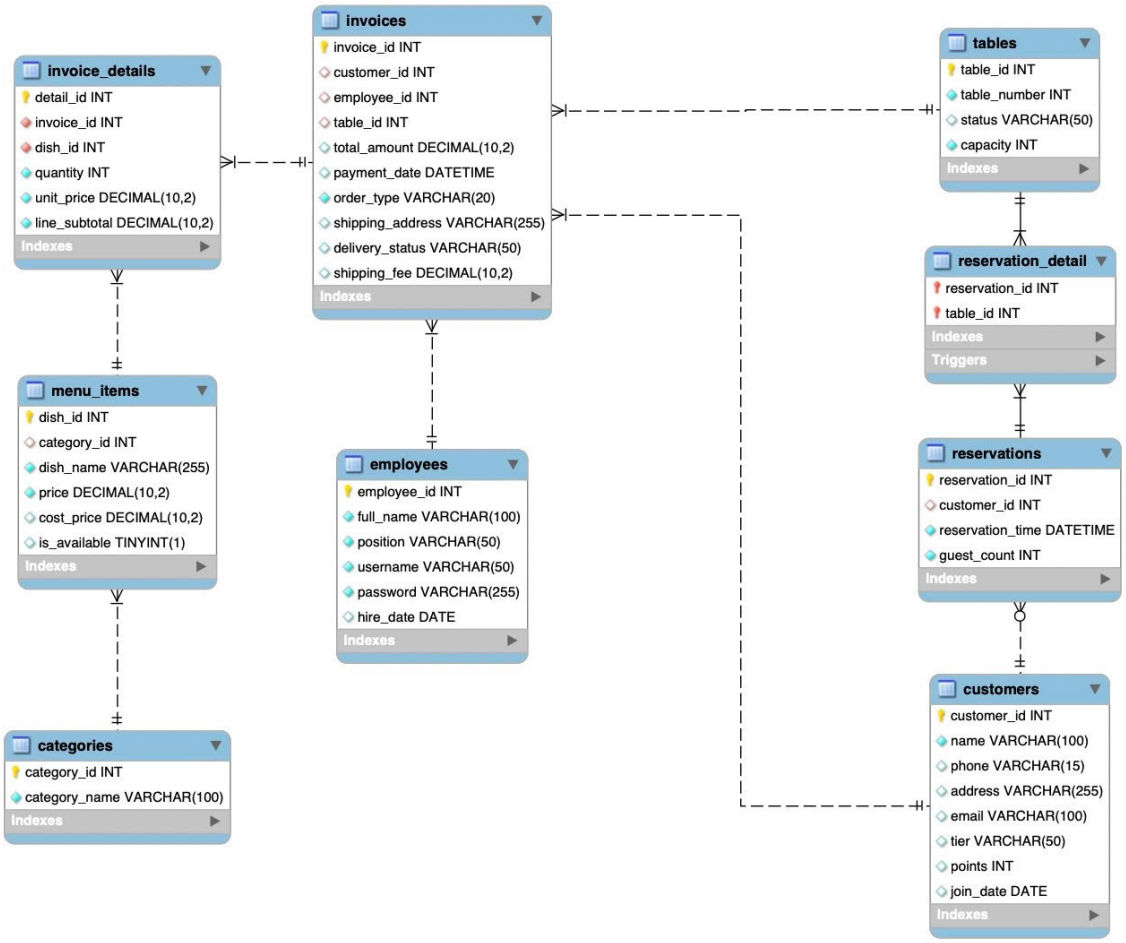

Figure 2.1: Entity-Relationship Diagram of the Restaurant Management System

2.3 Relational Schema Mapping

To transition from the conceptual ERD to a physical relational database, the entities were mapped into tables with clearly defined Primary Keys (PK) and Foreign Keys (FK). A critical part of this phase was resolving Many-to-Many (M:N) relationships to ensure database normalization (minimizing data redundancy and avoiding insertion/update anomalies). Two bridging (composite) tables were introduced:

- `reservation_detail` **Table:** Resolves the M:N relationship between `reservations` and `tables`. Its composite primary key consists of `(reservation_id, table_id)`.
- `invoice_details` **Table:** Resolves the M:N relationship between `invoices` and `menu_items`. It uses `detail_id` as the primary key while referencing both the invoice and the specific dish.

Furthermore, Referential Integrity Constraints were strictly applied. For bridging tables (`invoice_details` and `reservation_detail`), the `ON DELETE CASCADE` action was

implemented to automatically clear related details if an invoice or reservation is deleted. Conversely, for entities like `employees` or `customers`, the `ON DELETE SET NULL` action was used in the `invoices` table to preserve historical financial records even if a staff member or customer profile is removed.

2.4 Table Structures and Constraints

The final database schema consists of 9 integrated tables. Below is the Data Dictionary outlining the structure, data types, and constraints of the core tables:

Table 2.1: Data Dictionary for `employees` table

Attribute Name	Data Type	Key/Constraint	Description
employee_id	INT	PK, Auto Increment	Unique identifier for each staff member
full_name	VARCHAR(100)	NOT NULL	Full name of the employee
position	VARCHAR(50)	NOT NULL	Job role (e.g., admin, cashier, waiter)
username	VARCHAR(50)	UNIQUE, NOT NULL	Login username
password	VARCHAR(255)	NOT NULL	SHA-256 hashed password
hire_date	DATE		Date of employment

Table 2.2: Data Dictionary for `customers` table

Attribute Name	Data Type	Key/Constraint	Description
customer_id	INT	PK, Auto Increment	Unique identifier for customers
name	VARCHAR(100)	NOT NULL	Full name of the customer
phone	VARCHAR(15)		Contact number
address	VARCHAR(255)		Home or delivery address
email	VARCHAR(100)		Email address
tier	VARCHAR(50)	DEFAULT 'Standard'	Membership level
points	INT	DEFAULT 0	Accumulated loyalty reward points
join_date	DATE	DEFAULT CURRENT_DATE	Account creation date

Table 2.3: Data Dictionary for `tables` table

Attribute Name	Data Type	Key/Constraint	Description
table_id	INT	PK, Auto Increment	System ID for the table
table_number	INT	UNIQUE, NOT NULL	Physical table number
status	VARCHAR(50)	DEFAULT 'Available'	Current state (Available/Reserved)
capacity	INT	NOT NULL	Maximum seating capacity

Table 2.4: Data Dictionary for `categories` table

Attribute Name	Data Type	Key/Constraint	Description
category_id	INT	PK, Auto Increment	Unique category ID
category_name	VARCHAR(100)	UNIQUE, NOT NULL	Name of the category (e.g., Drinks)

Table 2.5: Data Dictionary for `menu_items` table

Attribute Name	Data Type	Key/Constraint	Description
dish_id	INT	PK, Auto Increment	Unique dish ID
category_id	INT	FK	Links to categories table
dish_name	VARCHAR(255)	NOT NULL	Name of the dish
price	DECIMAL(10,2)	NOT NULL	Selling price
cost_price	DECIMAL(10,2)		Original cost for profit analysis
is_available	TINYINT(1)	DEFAULT 1	Stock availability flag (1=Yes, 0=No)

Table 2.6: Data Dictionary for `reservations` table

Attribute Name	Data Type	Key/Constraint	Description
reservation_id	INT	PK, Auto Increment	Unique booking ID
customer_id	INT	FK	Links to customers table
reservation_time	DATETIME	NOT NULL	Scheduled time for the booking
guest_count	INT	NOT NULL	Number of arriving guests

Table 2.7: Data Dictionary for `reservation_detail` table

Attribute Name	Data Type	Key/Constraint	Description
reservation_id	INT	PK, FK	Links to reservations table
table_id	INT	PK, FK	Links to tables table

Table 2.8: Data Dictionary for `invoices` table

Attribute Name	Data Type	Key/Constraint	Description
invoice_id	INT	PK, Auto Increment	Unique transaction ID
customer_id	INT	FK	Links to customers table (Nullable)
employee_id	INT	FK	Identifies the staff who created the invoice
table_id	INT	FK	Links to tables table for Dine-in orders (Nullable)
total_amount	DECIMAL(10,2)	DEFAULT 0.00	Final calculated bill amount
payment_date	DATETIME		Timestamp when payment was completed
order_type	VARCHAR(50)	NOT NULL	Type of order (e.g., Dine-in, Takeaway, Delivery)
shipping_address	VARCHAR(255)		Delivery address for external orders (Nullable)
delivery_status	VARCHAR(50)	DEFAULT 'Pending'	Real-time status of the delivery
shipping_fee	DECIMAL(10,2)	DEFAULT 0.00	Delivery fee (if applicable)

Table 2.9: Data Dictionary for `invoice_details` table

Attribute Name	Data Type	Key/Constraint	Description
detail_id	INT	PK, Auto Increment	Unique identifier for the invoice line item
invoice_id	INT	FK	Links to invoices table
dish_id	INT	FK	Links to menu_items table
quantity	INT	NOT NULL	Quantity of the dish ordered
unit_price	DECIMAL(10,2)	NOT NULL	Price of the dish at the time of purchase
line_subtotal	DECIMAL(10,2)	NOT NULL	Computed as quantity × unit_price

2.5 Sample Data Generation

To simulate real-world operations and test the validity of the database constraints, Data Manipulation Language (DML) scripts were utilized. Between 5 to 10 sample rows were inserted into each table. This mock data includes diverse scenarios, such as walk-in guests, VIP customers (Platinum tier), various menu categories, and multi-item invoices.

Below is an excerpt of the SQL script used to generate sample data for the `customers` and `menu_items` tables:

```
-- Inserting sample data into customers table
INSERT INTO customers (name, phone, address, email, tier, points,
    join_date) VALUES
```

```
('Nguyen Ha An', '0901234567', '123 Le Loi', 'a@gmail.com', 'Gold', 150, '2023-01-10'),
('Tran Thi Hang', '0912345678', '456 Tran Hung Dao', 'b@gmail.com', 'Standard', 20, '2025-02-15'),
('Le Tung Duong', '0923456789', '789 Nguyen Hue', 'c@gmail.com', 'Platinum', 300, '2024-03-20');

-- Inserting sample data into menu_items table
INSERT INTO menu_items (category_id, dish_name, price, cost_price, is_available) VALUES
(1, 'Grilled Chicken Sandwich', 75000, 35000, 1),
(1, 'Beef Burger with Fries', 95000, 45000, 1),
(9, 'Beef Noodle Soup', 70000, 25000, 1),
(10, 'Pepperoni Pizza', 240000, 60000, 1);
```

Chapter 3

Advanced Database Implementation

3.1 Advanced SQL Features

To enhance automation, abstract data handling, and implement embedded business logic directly within the database management system, several advanced SQL objects were programmed. This server-side implementation reduces application overhead and enforces data integrity across all operational channels.

3.1.1 Triggers for Auto-updating Table Status

A row-level database trigger named `after_reservation_table_insert` was engineered on the `reservation_detail` junction table. From an operational workflow perspective, a manual application step should not be responsible for flipping table availability states, as programmatic delays or errors could lead to double-booking anomalies. Under this triggered implementation, the moment a junction record successfully links a specific table to an active booking identifier, an implicit state change is fired. The database engine automatically updates the target table's state field within the `tables` storage partition to 'Reserved'.

```
DELIMITER //
CREATE TRIGGER after_reservation_table_insert
AFTER INSERT ON reservation_detail
FOR EACH ROW
BEGIN
    -- Update table status based on the inserted table_id
    UPDATE tables
    SET status = 'Reserved'
    WHERE table_id = NEW.table_id;
END //
DELIMITER ;
```

3.1.2 Views for Top-selling Dishes

A database View named `View_TopSellingDishes` was implemented as a virtual relational table to simplify complex analytical queries. Computing item performance statistics natively requires executing structural `JOIN` statements across both the inventory master files

(menu_items) and individual transaction rows (invoice_details), combined with computation expressions (SUM). By encapsulating this query logic into an optimized database View, the underlying system abstracts the raw table joins away from application components.

```
CREATE VIEW View_TopSellingDishes AS
SELECT m.dish_name, IFNULL(SUM(id.quantity), 0) as total_sold
FROM menu_items m
LEFT JOIN invoice_details id ON m.dish_id = id.dish_id
GROUP BY m.dish_name
ORDER BY total_sold DESC;
```

3.1.3 Stored Procedures and User Defined Functions (UDFs)

To minimize excessive cloud-network roundtrips, business computational logic was localized directly on the database server using Stored Procedures and User Defined Functions.

First, a Stored Procedure named CalculateInvoiceTotal was created. This procedure aggregates line subtotals and calculates the final invoice amount (including shipping fees) directly over the server's data structures, updating the targeted invoice entity using a single integer input parameter.

```
DELIMITER //
CREATE PROCEDURE CalculateInvoiceTotal(IN inv_id INT)
BEGIN
    UPDATE invoices
    SET total_amount = (SELECT IFNULL(SUM(line_subtotal), 0)
                        FROM invoice_details
                        WHERE invoice_id = inv_id) + IFNULL(
                           shipping_fee, 0)
    WHERE invoice_id = inv_id;
END //
DELIMITER ;
```

Second, a User Defined Function (UDF) named CalculateLoyaltyPoints was implemented to automate customer reward calculations. This function takes the total invoice amount as input and returns an integer representing the loyalty points earned (e.g., 1 point for every 100,000 VND spent). This output can then be used to instantly update the customer's profile.

```
DELIMITER //
CREATE FUNCTION CalculateLoyaltyPoints(total_amount DECIMAL(10,2)
   )
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE points INT;
    SET points = FLOOR(total_amount / 100000);
    RETURN points;
END //
DELIMITER ;
```

3.2 Database Security and Administration

Database security measures were integrated at both the structural engine tier and the logical programmatic interface tier to enforce data privacy, prevent privilege escalation, and safeguard structural storage layers.

3.2.1 User Roles and Privileges

Direct application connections via standard system administrator accounts represent a major vulnerability. To adhere strictly to the Principle of Least Privilege (PoLP), a dedicated operational security group profile named `staff_role` was declared. Relational data control scripts apply clear boundaries to this profile, granting strictly required data modification privileges (`SELECT`, `INSERT`, `UPDATE`) while explicitly omitting structural administration commands.

```
CREATE ROLE IF NOT EXISTS 'staff_role';
GRANT SELECT, INSERT, UPDATE ON RestaurantManagement.* TO '
    staff_role';
```

3.2.2 Data Security and Password Hashing (SHA-256)

Storing staff authentication access tokens or passwords in clear-text configurations constitutes a catastrophic security threat. To resolve this, native server-side encryption was coupled with backend client validation logic. Relational data seeding operations utilize the cryptographic hash standard `SHA2` at a bit length configuration of 256. To complement this, the Python frontend integrates an exact mirrored algorithmic hashing implementation utilizing the standardized `hashlib.sha256` module, achieving a secure, zero-knowledge verification flow.

3.2.3 Preventing SQL Injection

Dynamic query generation via raw string concatenations leaves backend parsing systems vulnerable to malicious SQL Injection payloads. This risk was completely eliminated by implementing Parameterized Queries throughout the programmatic interface tier. By deploying structured bind-variable syntax (`%s`), raw input texts are never evaluated directly as executable database logic commands, completely neutralizing script execution strings.

3.3 Performance Tuning and Indexing

As analytical transactional tables accumulate massive volumes of operational data rows over time, scanning whole tables sequentially results in significant disk I/O processing bottlenecks and increases statement response times. To achieve sub-second execution intervals across high-frequency transactional data lines, database indexing structures were created:

- `idx_dish_name`: Configured on the text segment attribute `dish_name` of the `menu_items` table to optimize execution times during dynamic searches.

- `idx_reservation_time`: Configured over temporal data logs within the `reservations` dataset to accelerate historical date-range parsing workloads.

These indexes significantly increase index selectivity, allowing the DBMS to quickly retrieve a small subset of rows from large tables without scanning the entire dataset. This greatly reduces the response time of end-user queries and ensures optimal database performance.

3.4 Backup and Recovery Protocols

Because system operations are deployed directly over an enterprise cloud framework through **Aiven Cloud**, business continuity and data durability are guaranteed through hardware-managed resilience features. The underlying cloud storage engine operates with strict continuous recovery logging pathways (Write-Ahead Logging - WAL). Relational snapshots and data states are backed up automatically at predefined automated intervals, allowing precise point-in-time recovery processes to safely re-stage stable database points if unexpected system crashes or cluster disruptions occur.

Chapter 4

Python Application Development

4.1 Application Architecture and Database Connection

The Restaurant Management System is divided into two main parts: a MySQL database for storing data and a Streamlit web interface for users. We use the `mysql.connector` library in Python to connect our web app to the remote Aiven Cloud database.

To manage the connection safely and easily, we created a function named `get_db_connection()`. This function reads the database credentials (like host, username, and password) from the `DB_CONFIG` dictionary and creates a secure connection every time the app needs to interact with the database.

```
# --- DATABASE CONFIGURATION ---
DB_CONFIG = {
    "host": "mysql-3c1b790c-vudieulinh305-ebb6.k.aivencloud.com",
    "port": 25428,
    "user": "avnadmin",
    "password": "AVNS_AIQ1h70s2tSBuu4XrKE",
    "database": "RestaurantManagement"
}

def get_db_connection():
    try:
        return mysql.connector.connect(**DB_CONFIG, ssl_disabled=
           False)
    except Exception as e:
        st.error(f"Connection Error: {e}")
        return None
```

4.2 Data Operations and Transaction Management (ACID)

When performing complex operations like checking out and generating an invoice, the system must update multiple tables simultaneously. If a network error happens in the middle of this process, the database could become corrupted or incomplete.

To prevent this and ensure ACID compliance (data safety), we heavily utilize database transactions. A prime example is our new **Checkout and Loyalty Point Redemption** module. When a bill is finalized, the system must: (1) Create the invoice record, (2) Insert multiple order items into the `invoice_details` table, and (3) Update the customer's loyalty points balance dynamically.

We start by calling `conn.start_transaction()`. If all database operations are successful, we save the changes permanently using `conn.commit()`. If any error occurs, the system catches it and calls `conn.rollback()` to revert all temporary changes.

```
# --- TRANSACTION MANAGEMENT FOR CHECKOUT ---
if st.button("Generate Invoice & Checkout"):
    try:
        cursor = conn.cursor()
        conn.start_transaction()

        # 1. Insert into Invoices table
        cursor.execute("INSERT INTO invoices (customer_id,
           total_amount, payment_date, order_type,
           delivery_status) VALUES (%s, %s, %s, %s, 'Delivered')"
           , (customer_id, final_total, datetime.now(),
           order_type))
        new_invoice_id = cursor.lastrowid

        # 2. Insert into Invoice_Details table
        for item in order_items:
            cursor.execute("INSERT INTO invoice_details (
               invoice_id, dish_id, quantity, unit_price,
               line_subtotal) VALUES (%s, %s, %s, %s, %s)", (
               new_invoice_id, item['dish_id'], item['quantity'],
                item['price'], line_subtotal))

        # 3. Update Customer Reward Points
        if customer_id:
            cursor.execute("UPDATE customers SET points = points
               - %s + %s WHERE customer_id = %s", (
               points_to_redeem, earned_points, customer_id))

        conn.commit() # Save changes permanently
        st.success("Invoice created successfully!")

    except Exception as e:
        conn.rollback() # Cancel all changes if error occurs
        st.error(f"Error creating invoice: {e}")
```

4.3 Interactive User Interface

Streamlit is a great tool for building web apps quickly, but it automatically reloads the script every time a user interacts with the page (like clicking a button). To prevent the app from losing data during reloads, we use `st.session_state` to remember important information.

We use `st.session_state` and Streamlit components for several core features:

- **User Authentication:** It remembers the login status and the user's role (`admin`, `cashier`, `waiter`) after they log in successfully.
- **Secure Password Management:** For security purposes, staff cannot register accounts freely. However, they can securely update their assigned passwords. The system verifies their old password, checks if the new password meets the 6-character length requirement, ensures passwords match, and then hashes the new password using SHA-256 before updating the database.
- **Dynamic Ordering:** When creating an invoice, we use `st.session_state.item_count` to count how many dishes the customer wants. Every time the "Add Another Dish" button is clicked, this number increases, and the web page dynamically adds a new input row.
- **Loyalty Point Redemption Logic:** During checkout, the interface dynamically fetches a customer's available reward points. It automatically calculates the maximum allowable discount and updates the grand total and newly earned points in real-time before finalizing the transaction.
- **Admin Dashboard:** It helps render interactive charts (using `st.line_chart` and `st.bar_chart`) to show daily revenue and top-selling dishes for administrators.

4.4 Cloud Deployment System

To make the application available online for restaurant staff to use anywhere, we deployed it to the cloud using a simple two-step process:

- **GitHub:** We pushed our project files, including the main Python script (`dbms.py`) and the required libraries (`requirements.txt`), to a public GitHub repository.
- **Streamlit Cloud:** We connected Streamlit Cloud to our GitHub repository. Streamlit automatically reads the code, installs the necessary packages, and hosts the web application on a public URL. This allows the system to run 24/7 without needing a local server.

Chapter 5

Testing and Results

5.1 Test Cases

To ensure that our Restaurant Management System works correctly and safely, we created and executed multiple test cases. These test cases focus on testing core business operations, validation rules, and transactional security (ACID).

5.1.1 Test Case 1: Staff Authentication (Login)

This test case verifies if the login system correctly identifies valid credentials, handles invalid attempts, and allocates the correct user role context using `st.session_state`.

Table 5.1: Test Case Matrix for Staff Login

ID	Scenario	Input Data	Expected Result	Status
TC-1.1	Successful Admin Login	Username: 'admin' Password: '123456'	Access granted. App opens with Admin Sidebar menu option.	Pass
TC-1.2	Successful Waiter Login	Username: 'waiter1' Password: '123456'	Access granted. App opens with Waiter Sidebar (No Admin Reports).	Pass
TC-1.3	Failed Login (Wrong Password)	Username: 'admin' Password: 'wrong_pass'	Access denied. Shows error message: "Invalid credentials."	Pass

5.1.2 Test Case 2: Table Booking and Capacity Validation

This test case verifies if the booking function correctly checks the seating capacity of the table before confirming a reservation, and ensures the table status changes to 'Reserved'.

Table 5.2: Test Case Matrix for Table Reservations

ID	Scenario	Input Data	Expected Result	Status
TC-2.1	Valid Booking	Select Table 1 (Capacity: 2) Guests: 2	Booking confirmed. Table status changes from 'Available' to 'Reserved'.	Pass
TC-2.2	Invalid Booking (Over Capacity)	Select Table 1 (Capacity: 2) Guests: 4	Booking blocked. Error message: "Cannot book! Table only has a capacity of 2."	Pass

5.1.3 Test Case 3: Invoice Generation and Transaction Security

This testcase ensures that creating a checkout invoice correctly updates the bill amounts and verifies the rollback feature if a system error occurs during the multi-row insert.

Table 5.3: Test Case Matrix for Checkout Operations

ID	Scenario	Input Data	Expected Result	Status
TC-3.1	Multi-item Bill Calculation	Item 1: Qty 2 × 75,000 Item 2: Qty 1 × 100,000	Invoice created. Grand Total shows exactly 250,000 VND.	Pass
TC-3.2	Transaction Safety (Rollback Test)	Input missing client ID or force database disconnect mid-way.	System aborts execution. Rollback triggers. No partial or corrupted invoice details saved.	Pass

5.2 Application Screenshots

This section presents actual graphical interface screenshots of our functioning system hosted live on Streamlit Cloud, illustrating successful operations.

5.2.1 System Authentication Interface

The following screenshot shows the active web secure login panel used by restaurant staff members to enter the application environment.

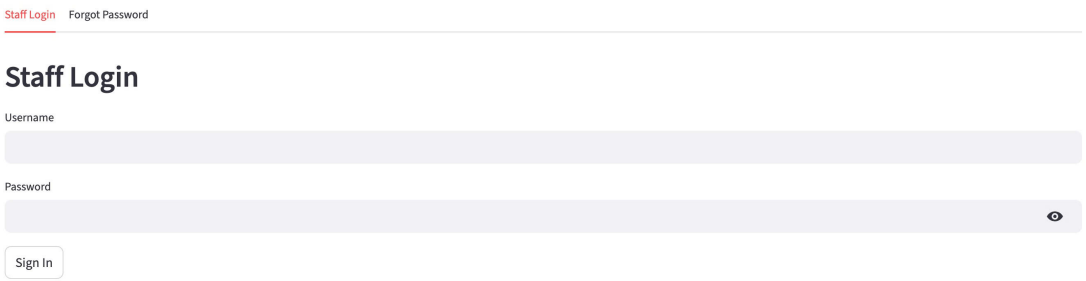

Figure 5.1: Live Web Interface for Staff Login Panel

5.2.2 Security and Password Management

To enforce operational security, an internal password management module allows authorized staff to securely update their credentials. The system validates the old password and enforces character limits before hashing the new string.

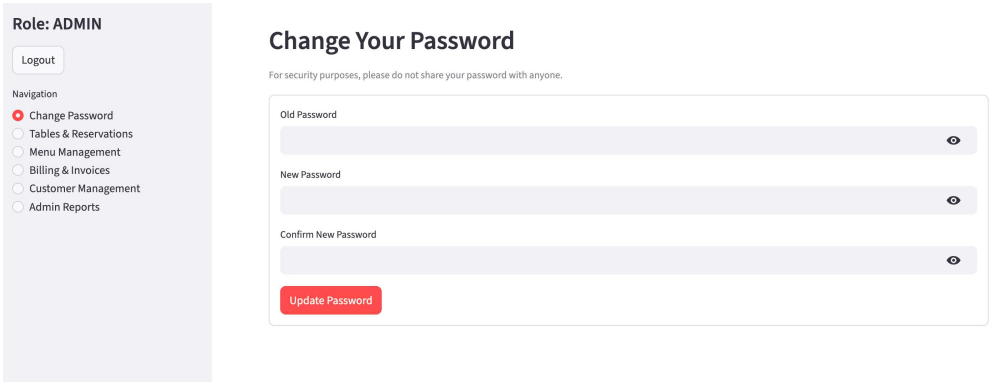

Figure 5.2: Secure Password Management Module

5.2.3 Table and Reservation Management Screen

This screen displays the layout status of restaurant tables and allows users to input guest configurations to make real-time bookings.

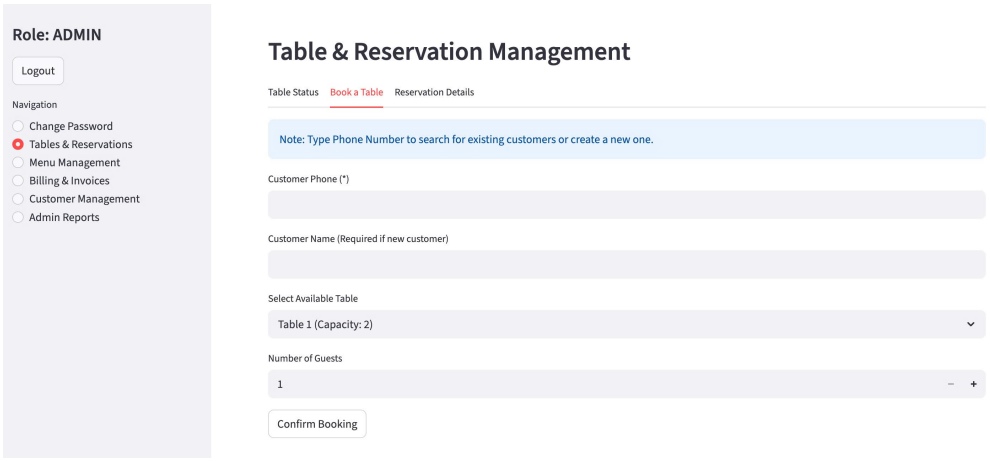

Figure 5.3: Live Seating Layout and Reservation Validation Module

5.2.4 Menu Management Screen

The digital dish repository partition where authorized staff can monitor dish prices, underlying cost prices, and update real-time stock availability.

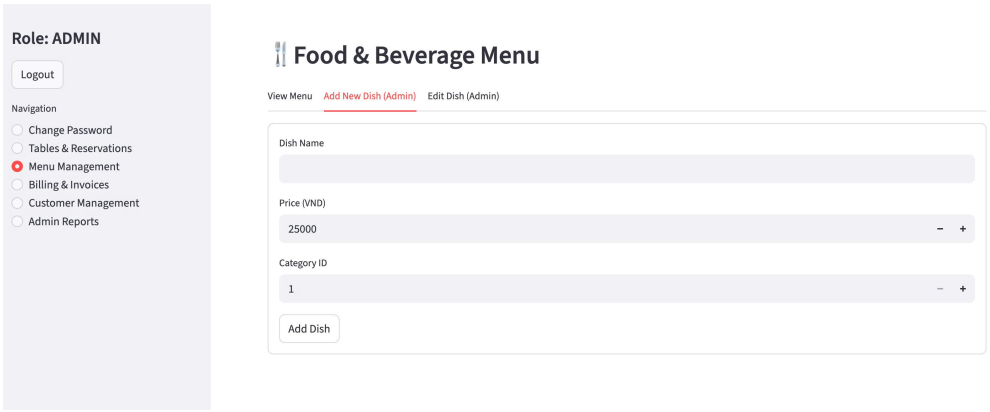

Figure 5.4: Live Restaurant Menu Master File and Dish Availability Monitor

5.2.5 Billing and Dynamic Checkout Module

The final checkout utility demonstrating the nested dynamic order rows and automated calculation fields before final database commit operations.

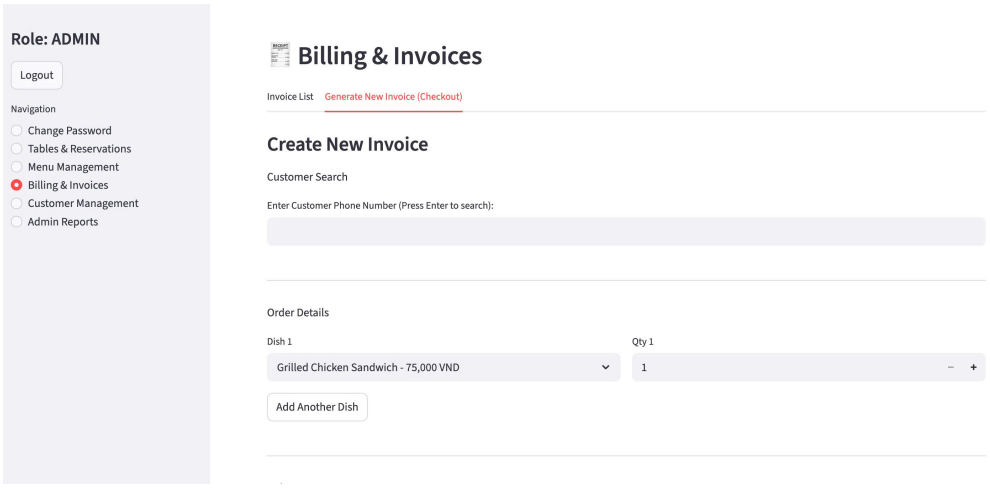

Figure 5.5: Live Dynamic POS Checkout and Invoice Generation Utility

5.2.6 Customer Management Screen

The customer dashboard showing the comprehensive list of registered guests dynamically ordered by their accumulated loyalty reward points and membership tiers.

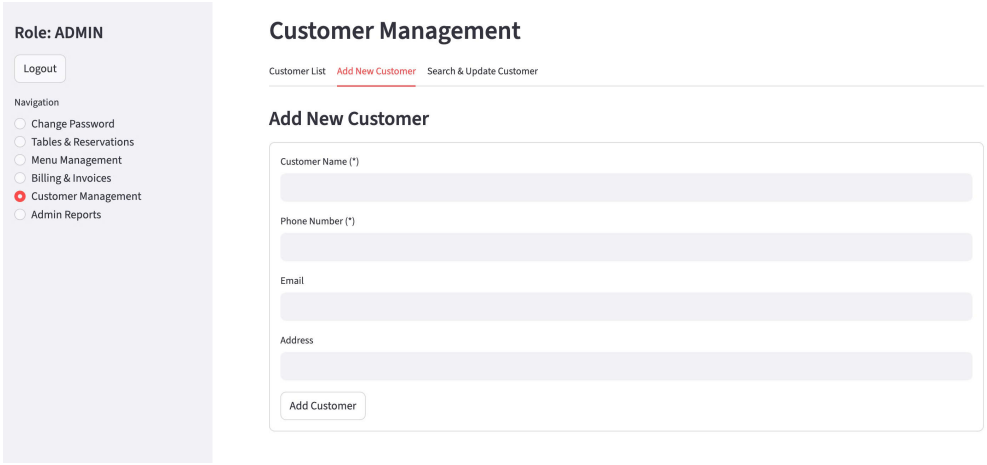

Figure 5.6: Live Customer Profile Database and Loyalty Tier System

5.2.7 Admin Analytical Dashboard Portals

Restricted graphical canvas accessible only to accounts with administrative permission privileges, showing dynamic charts drawn from cloud view partitions.

Figure 5.7: Live Analytical Portal and Interactive Revenue Tracking Graphs

Chapter 6

Conclusion and Recommendations

6.1 Summary of Project Results

In conclusion, our group has successfully designed, built, and deployed a live Restaurant Management System. By combining a relational database with a modern web interface, we achieved the following results:

- **Robust Database Design:** We designed a standardized database schema with 9 tables. By using junction tables like `reservation_detail` and `invoice_details`, we successfully resolved all many-to-many relationships and minimized data redundancy.
- **Advanced SQL Features:** We successfully implemented database triggers to automate table status changes, created virtual views for sales reports, and stored procedures to handle complex invoice calculations on the server side.
- **Secure and Interactive Application:** We created a web-based interface using Python and Streamlit. The application is completely secure against SQL Injection thanks to parameterized queries, and protects sensitive user data using SHA-256 password hashing.
- **Integrated Loyalty Program:** We successfully engineered a dynamic customer loyalty system that tracks reward points, allows point redemption for discounts, and updates profiles automatically via ACID transactions.
- **Cloud Deployment:** The system is hosted entirely on Aiven Cloud (for the MySQL database) and Streamlit Cloud (for the web application). This setup allows restaurant staff to concurrent access and update data from any device with an internet connection.

6.2 System Limitations

Although the system meets all the project requirements, it still has a few limitations that can be improved:

- **Simple Authentication:** The system currently uses a basic login form. It does not support advanced security features like session timeouts or multi-factor authentication (MFA).

- **No Real-time Notification:** When a table status changes or a new reservation is created, the system updates the database instantly, but it does not send real-time alerts or emails to the restaurant managers or customers.
- **Stateless Page Reloads:** Because of how Streamlit works, the entire web page reloads whenever a user inputs data. Even though we use `st.session_state` to save critical data, this reloading behavior can sometimes slow down the user experience.

6.3 Future Enhancements

To make the application more practical for commercial use, we recommend the following future developments:

- **Online Ordering System:** Expand the application into a customer-facing website where guests can browse the food menu, customize order configurations, and place online takeaway or delivery orders directly.
- **QR-Code Table Check-in:** Integrate unique QR codes for each dining table. Customers can scan the QR code with their mobile phones to view the dynamic menu, call waiters, or pay their invoices digitally without waiting for a physical bill.
- **Advanced Security and Monitoring (IP Whitelisting & Audit Logs):** Although the current system implements robust role-based access control and SHA-256 password hashing, future iterations will integrate comprehensive database Audit Logs. These logs will automatically record and track all data manipulation operations (INSERT, UPDATE, DELETE) per user, enabling administrators to easily pinpoint access violations and track staff activities. Furthermore, network-level security will be upgraded by transitioning from open cloud access to strict IP Whitelisting. This ensures the Aiven Cloud database will only accept connections from authorized network environments, such as the restaurant's internal Wi-Fi or the Streamlit hosting server.

References

[1] Carlos Coronel, Steven Morris, *Database Systems: Design, Implementation, & Management*, 14th ed., Cengage Learning, 2023.

[2] Streamlit Framework Documentation, https://docs.streamlit.io

Appendix A

Project Links and Resources

- Live Web Application
- GitHub Repository (Source Code)
- Project Presentation Video
- System Live Demo Video